



UNSW
SYDNEY

COMP4141

Theory of Computation

Course Recap

Outline

Final exam details

Course recap (Weeks 1-6)

Course recap (Weeks 6-10)

Sample questions

Final exam

Final exam:

- Wednesday, May 3
- Available 10am AEST
- **Due: Thursday, May 4, 10am AEST**
- Submission via give: multiple submissions ok - last one before due date counts
- Lateness: 25% per 15 minutes or part thereof.
- Worth 50% of overall grade. Must achieve 40% to pass course.

Final exam

- 10 questions:
 - 6-7 questions on Week 1-5 content;
 - 3-4 questions on Week 7-10 content
- Proof-based questions
- Assessment based on **understanding** and **ability**
 - Any **passive** (i.e. existing) material acceptable – properly referenced
 - Any material taken from **active** sources is in breach of Student Conduct and will be subject to plagiarism review
 - University policy: ChatGPT is considered an active source
- Relevant announcements made via webCMS/email within first 2 hours. Issues after this should be self-resolved.

Special consideration

Review the UNSW policy on [Special Consideration](#).

UNSW has a “fit-to-sit” policy: by undertaking the assessment you are declaring that you are fit to do so.

If there are any foreseeable issues you must apply before the exam. If circumstances prevent you from applying before the exam, you must apply as soon as possible (within 3 days).

NB

Supplementary exams are only available to students granted Special Consideration.

Technical issues

If you experience technical issues before or during the exam, you should follow these instructions:

- Take screenshots (including time and date) of as many of the following as possible:
 - error messages
 - screen not loading
 - timestamped speed tests
 - power outage maps
 - messages or information from your internet provider regarding the issues experienced
- Make contact with me immediately (through email or ed) and advise me of the issue.
- Submit a Special Consideration application immediately at the conclusion of your assessment and upload your screenshots.

Outline

Final exam details

Course recap (Weeks 1-6)

Course recap (Weeks 6-10)

Sample questions

Weeks 1-2 recap

- Formal languages
 - Alphabets, Strings/words, Languages
 - Recursive definitions
- Deterministic Finite Automata (DFA)
 - $(Q, \Sigma, \delta, q_0, F)$
 - $L(A)$
- Non-deterministic Finite Automata (NFA)
 - Non-determinism
 - ϵ -transitions
- Equivalence of DFAs and NFAs
 - Subset construction
- Regular expressions
 - Definition, $L(R)$
 - Equivalence to DFAs
- Pumping Lemma
- Myhill-Nerode theorem

Need to know

- Regular expressions
- DFAs, NFAs, Regular Expressions: Definitions, how to use
 - For example: Finding A or L such that $L = L(A)$
- Formally arguing correctness
- Pumping Lemma - how to use it, limitations
- Myhill-Nerode theorem - how to use it, limitations

Weeks 3-4 recap

Context-free languages

- Context-free grammars
 - Derivations, parse trees, ambiguity
- Pushdown automata
- Pumping lemma for CFLs
- Closure properties of CFLs
- Chomsky normal form
- Non-emptiness and word acceptance algorithms
 - CKY algorithm
- Chomsky hierarchy

Need to know

- CFGs and PDAs: Definitions, how to use
- Pumping lemma: How to use
- Algorithms for CFLs

Weeks 4-6 recap

Turing Machines, Recursive and Recursively Enumerable languages

- Turing Machines
 - Definitions, accept vs decide
 - Variants: stay-put, semi-infinite tape, multi-tape, non-determinism
 - Enumerators
 - Universal Turing Machine
- **Rec, RE, coRE**
 - **$\text{Rec} = \text{RE} \cap \text{coRE}$**
- Encodings
- Reductions: many-one vs Turing
- Undecidable problems: $A_{TM} = \{\langle M, w \rangle : M \text{ accepts } w\}$
- Rice's theorem

Need to know

- TMs: Definitions, how they work
- Many-one reductions: Definitions, how to use
- Oracles
- Rice's theorem: When it applies

Outline

Final exam details

Course recap (Weeks 1-6)

Course recap (Weeks 6-10)

Sample questions

Weeks 6-7 recap

- Resource-bounded computation:
 - **TIME** $t(n)$
 - **NTIME** $t(n)$
 - **SPACE** $t(n)$
 - **NSPACE** $t(n)$
- Time and Space-hierarchy theorems
- **P** and **NP**
- **NP**-completeness

Need to know

- How to use the Space/Time-hierarchy theorems
- Class definitions and relation to other classes
- How to do **NP**-completeness proofs

Weeks 7-8 recap

NP-completeness proofs

- SAT (Cook's theorem)
- 3SAT, 4SAT
- VERTEX COVER
- CLIQUE
- HAMILTONIAN PATH, HAMILTONIAN CYCLE
- TRAVELLING SALESMAN
- SUBSET SUM
- INTEGER LINEAR PROGRAMMING, 0-1 ILP
- 3COL

Need to know

Reduction strategies for **NP**-completeness proofs

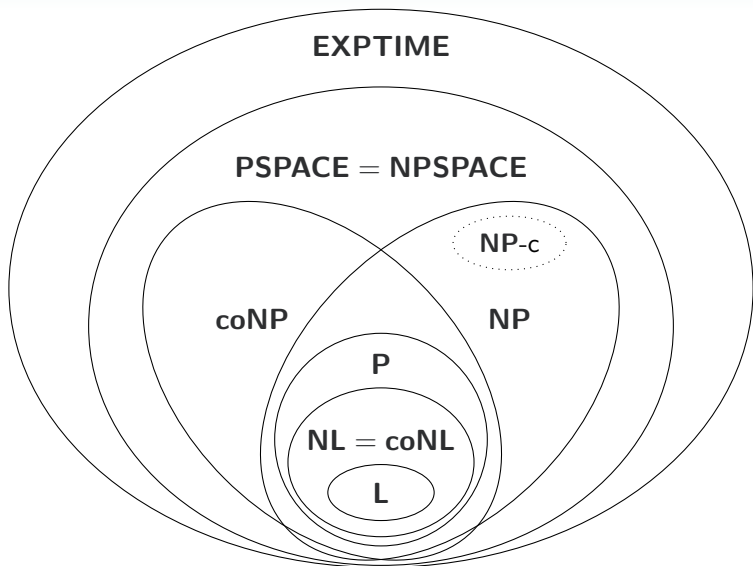
- Generalization
- Problem restatement
- Gadget construction

Weeks 8-9 recap

Complexity classes:

- **coNP**, VALIDITY
- **EXPTIME**, BOUNDED HALT
- **PSPACE**, QBF (QSAT)
- **L**, **NL**, $\leq_L$, PATH

Complexity classes



Complexity summary

Class	Characterization	Problem	Notes
P	$\bigcup_{k \in \mathbb{N}} \text{TIME}(n^k)$	HORN-SAT	
NP	$\bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$	SAT, 3SAT	Poly. verifiable
PSPACE	$\bigcup_{k \in \mathbb{N}} \text{SPACE}(n^k)$	QBF	= NPSPACE
coNP	Languages L such that $\bar{L} \in \text{NP}$	VALIDITY	Poly. refutable
EXPTIME	$\bigcup_{k \in \mathbb{N}} \text{TIME}(2^{n^k})$	BOUNDED-HALT	$\neq \text{P}$
L	SPACE (log n)	UND. PATH	
NL	NSPACE (log n)	PATH, 2SAT	= coNL

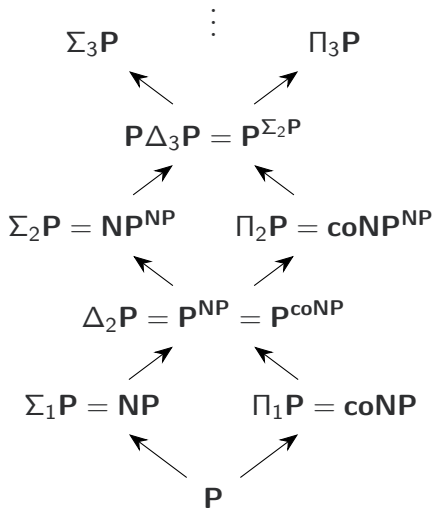
Need to know

- Class definitions and relation to other classes
- Consequences of Savitch's theorem
- **PSPACE**-completeness proofs
- How to do logspace reductions

Weeks 9-10 recap

- Oracle machines
- Alternation
- Polynomial-time hierarchy
- Probabilistic computation
 - Probabilistic Turing Machines
 - Probabilistic complexity classes: **PP**, **BPP**, **RP**, **ZPP**.

PSPACE



Need to know

- Oracle machines: Definition, how to use
- Alternation: Definition, how to use
- Probabilistic computation: Definition, how to use
- Relation between Alternation complexity classes and classical complexity classes
- Probabilistic complexity classes: **PP**, **BPP**, **RP**, **ZPP**

Outline

Final exam details

Course recap (Weeks 1-6)

Course recap (Weeks 6-10)

Sample questions

2022 Q6

Are the following decision problems decidable, recursively enumerable but undecidable, or not recursively enumerable? Justify your answers.

a

DFA INCLUSION

Input: A Turing Machine M and a DFA D

Question: Is $L(D) \subseteq L(M)$?

(5 marks)

Are the following decision problems decidable, recursively enumerable but undecidable, or not recursively enumerable? Justify your answers.

b

BPP MEMBERSHIP

Input: A Non-deterministic Turing Machine M that halts in polynomial time on all runs of all inputs

Question: Is it the case that for all inputs: either at least $\frac{2}{3}$ of all runs are accepting, or at least $\frac{2}{3}$ of all runs are rejecting?

(5 marks)

2022 Q7

The **Surveillance Problem** can be described as follows. A museum contains a number of rooms and can purchase a number of cameras to surveil these rooms. Each camera can surveil multiple rooms and rooms can be surveiled by multiple cameras (this information is given by a *monitoring relation*). The goal is to monitor all the rooms with as few cameras as possible. Formally stated as a decision problem:

SURVEILLANCE PROBLEM

Input: A set C of cameras; a set R of rooms; a monitoring relation $M \subseteq C \times R$; and an integer k

Question: Is there a subset $C' \subseteq C$ such that:

- $|C'| \leq k$, and
- $R = M(C') = \{r : (c, r) \in M \text{ for some } c \in C'\}$?

Show that the Surveillance Problem is **NP**-complete.

2022 Q8

In the sport of **Rogaining** the challenge is to visit checkpoints to accrue a score in a limited timeframe. Checkpoints that are more difficult to reach usually have a higher value. The goal of the competitors is to choose a route that maximizes the value of the checkpoints seen without exceeding the allocated time (not all checkpoints have to be visited, but you can only visit a checkpoint once). Formally, as a decision problem:

ROGAINING

Input: A set $V = \{v(1), \dots, v(n)\}$ of n checkpoint values; an $n \times n$ table $\{t(i, j)\}$ where $t(i, j)$ indicates the time to get from checkpoint i to checkpoint j ; an integer T ; and an integer V

Question: Is there a sequence $c_1, c_2, \dots, c_r$ where:

- $c_i \in [1, n]$ and $c_i \neq c_j$ for $i \neq j$;
- $\sum_{i=1}^{r-1} t(c_i, c_{i+1}) \leq T$; and
- $\sum_{i=1}^r v(c_i) \geq V$?

Show that the Rogaining Problem is **NP**-complete.

- a Show that the language

$$\{ww : w \in \{0,1\}^*\}$$

is in **L**. (5 marks)

- b Suppose that L is a language and let M be a probabilistic polynomial time Turing machine such that, for some fixed numbers $0 < \epsilon_1 < \epsilon_2 < 1$, we have
- $w \notin L$ implies that $P(M \text{ accepts } w) \leq \epsilon_1$, and
 - $w \in L$ implies that $P(M \text{ accepts } w) \geq \epsilon_2$.

Prove that $L \in \mathbf{BPP}$ (5 marks)

- a Show that the language

$$\{ww : w \in \{0,1\}^*\}$$

is in **L**.

(5 marks)

- b Suppose that L is a language and let M be a probabilistic polynomial time Turing machine such that, for some fixed numbers $0 < \epsilon_1 < \epsilon_2 < 1$, we have
- $w \notin L$ implies that $P(M \text{ accepts } w) \leq \epsilon_1$, and
 - $w \in L$ implies that $P(M \text{ accepts } w) \geq \epsilon_2$.

Prove that $L \in \mathbf{BPP}$

(5 marks)

– END OF COURSE –